University of Cape Town
ECO5040W — Financial Software Engineering

Business and Technical Specification

XRPL-Based FX Remittance Platform using RLUSD

Annita Ngoma Karabo Tigedi Kerry-Lynn Whyte
Liltha Mzamo Nikola Milosavljevic
Group 3

7 September 2026

This specification describes **Group 3's** XRPL remittance prototype as implemented in this GitHub fork (Karabo22Tigedi / FSE-Project). An earlier GitHub sketch by Kerry-Lynn Whyte established FastAPI, SQLite, Redis Streams, and Testnet Payments. This document is about *our* patched system, not that sketch. Cash-in and cash-out remain simulated. Using a dollar stablecoin does not remove South African exchange-control, anti-money-laundering, or licensing duties (University of Cape Town, 2026a). One hundred and thirteen automated tests currently pass.

1 Business problem

Cross-border remittances move household income to people who often collect cash at an agent. Western Union and MoneyGram are the familiar brands: the sender pays rand at an agent or online, the operator takes a fee, and the recipient collects cash on the other side after showing identification. Those operators are large, licensed money transmitters with agent networks in many countries. They are also expensive on small sends, and the price the customer sees is often a mix of a posted fee and a worse-than-mid-market exchange rate.

The World Bank's Remittance Prices Worldwide (RPW) survey is the standard public comparison of corridor costs (World Bank, 2025). South Africa is a high-cost sending market on that survey. Rateweb's 2026 reading of the 2025 Q1 RPW release reports Western Union and MoneyGram flat fees of about R214–R217 on a surveyed send of about R1,370 (Rateweb, 2026). That is a fee of roughly 15–16% of the send amount before any FX spread. For a family sending grocery money, that is a large bite.

Cost is not the only friction. The IMF's diagnostic of the South Africa–Zimbabwe corridor describes cash-heavy, agent-based payouts, delays, and weak price transparency (International Monetary Fund, 2025). FinMark Trust's 2024 assessment of South Africa to the rest of SADC likewise finds that cash collection and agent processes still dominate, and that customers often cannot see the full cost until they are already at the counter (FinMark Trust, 2024). Western Union and MoneyGram solve a real problem—getting cash to someone without a bank account—but they do it with many intermediaries, posted-plus-spread pricing, and a payout model that is slow to digitise.

Stablecoins such as RLUSD (a US-dollar token issued on the XRP Ledger) can move value between wallets in seconds once both sides hold the token. That does not magically replace the

cash-in door in Johannesburg or the cash-out door in Harare. The brief is explicit: a prototype may *simulate* those fiat legs (University of Cape Town, 2026a). Our platform therefore uses UCTUSD (Marc's course IOU) as the current *settlement* asset between a confirmed rand payment and a custodial recipient wallet; the API slot remains `rlusd_amount` and can later point at official RLUSD via `.env` if funded. We do not claim that putting value on a ledger deletes the South African Reserve Bank, FICA, or a remittance licence.

What we do claim, as a design, is narrower. A quoted fee of a fixed rand amount plus a percentage, plus a published FX margin on the conversion rate, is easier to audit than an opaque corridor tariff. A 15-minute quote with an explicit cancel path stops abandoned drafts from consuming a sender's daily limit. An on-chain Payment with a stored transaction hash gives the recipient something they can look up on a Testnet explorer. Those are the product bets this specification implements.

2 User journey

The journey follows the brief (University of Cape Town, 2026a) and is implemented end-to-end on the API (the web UI in Section 15 is planned, not yet built).

1. A customer registers (`POST /auth/register`) and logs in (`POST /auth/login`). The same customer account can send or receive; role is `customer` or `admin`, not two person types.
2. The sender submits mock KYC (`POST /kyc`). An admin approves or rejects it. Unapproved senders cannot create quotes.
3. The sender adds a beneficiary (`POST /beneficiaries`). If mobile or email already matches a registered user, the record links immediately; if the recipient registers later, linking happens then.
4. The sender posts a ZAR amount (`POST /remittances`). The API fetches USD/ZAR, applies the fee model, checks daily and monthly limits, and returns a quote with `tracking_ref` (`MG` plus ten digits) and `expires_at` (15 minutes).
5. The sender may cancel an unexpired quote (`POST /remittances/{id}/cancel`) or let it expire. Cancelled and expired quotes do not consume limits.
6. The sender initiates simulated cash-in (agent cash, bank transfer, or card). An admin confirms receipt. Settlement is not queued before that confirmation.
7. A Redis Streams worker submits an XRPL Testnet Payment from the platform treasury to the recipient's custodial account, stores the hash, and credits the internal spendable balance.
8. The recipient opens `GET /wallet/me`, sees spendable UCTUSD, incoming transfers with hashes, and cash-out history.
9. The recipient requests a cash-out to USD or ZAR. Fiat payout is still simulated, but complete submits a Payment of the reserved UCTUSD from the recipient custodial account to the official UCTUSD issuer (burn/redeem). `on_chain_balance` excludes completed cash-outs. Failed cash-outs refund spendable balance.

Anyone with a tracking reference who is the sender, the linked recipient, or an admin can call `GET /remittances/track/{ref}`.

3 Functional requirements

Table 1 maps the brief's required behaviour to *our* endpoints. Interactive OpenAPI lives at `/docs`. `GET /health` is unauthenticated.

Table 1: Functional behaviour mapped to this fork's API.

Capability	**Our implementation**
Register, login, logout, profile	`POST /auth/register`, `/auth/login`, `/auth/logout`; `GET`/`PATCH /users/me`. Passwords bcrypt; sessions hashed at rest.
Mock KYC + admin review	`POST /kyc`; `GET /kyc/me`, `/kyc/me/status`; admin `GET /kyc`, `POST /kyc/{id}/approve|reject`. ID number encrypted with the KYC Fernet key.
Beneficiaries	`POST`/`GET /beneficiaries`. Auto-link on mobile/email.
Quote, fees, limits	`POST /remittances`; `GET /limits/me`; admin `GET`/`PUT /admin/fee-config`, `GET`/`PUT /admin/limit-tiers/{tier}`.
Quote lifecycle	TTL 15 minutes; `POST /remittances/{id}/cancel`; cash-in rejected if expired or cancelled; fee-consumes-send rejected with 422.
Tracking	`tracking_ref` on `RemittanceOut`; `GET /remittances/track/{ref}`.
Simulated cash-in	`POST /remittances/{id}/cash-in`; admin `POST .../confirm-cash-in` enqueues settlement.
Queued XRPL settlement	Redis Stream `settlement_messages`; `POST /admin/settlement/run`; `GET /admin/settlement`; `POST /admin/settlement/{id}/retry` (pending, processing, or failed; completed is 409).
Custodial wallet	`GET /wallet/me`: `balance_rlusd` = spendable; also `spendable_balance` and `on_chain_balance` (excludes completed cash-outs).
Simulated cash-out	`POST /cash-outs`; `GET /cash-outs/me`; admin list/approve/complete/fail. USD and ZAR only.
Sender history	`GET /remittances/me`; admin `GET /remittances`.

4 Fee model

Fees are a single admin-editable `FeeConfig` row, not a history table. Percentages are fractions (0.0100 means 1%). Seeded defaults are a R25.00 fixed fee, 1% of send, 2% FX margin, and 1.5% cash-out.

Let `zar` be the send amount, `fixed` and `percentage` the transaction-fee parameters, `midmarket` the USD/ZAR rate, and `fx_margin` the spread fraction. Then:

$$\texttt{transaction\_fee} = \texttt{fixed} + \texttt{zar} \times \texttt{percentage}$$

$$\texttt{effective\_rate} = \texttt{midmarket} \times (1 + \texttt{fx\_margin})$$

$$\texttt{net\_zar} = \texttt{zar} - \texttt{transaction\_fee}$$

$$\texttt{rlusd} = \texttt{net\_zar} \,/\, \texttt{effective\_rate}$$

The quote is rejected (HTTP 422) if $\texttt{rlusd} \leq 0$ (equivalently if net ZAR after the transaction fee is not positive). Transaction fees are rounded to two decimal places; RLUSD amounts to six, half-up. RLUSD is treated as 1:1 with USD.

The FX margin makes the sender's conversion *worse* than mid-market: more rand per dollar means fewer dollars received. The cash-out percentage shown on the remittance quote is an *estimate*. At cash-out time the fee is reapplied to the RLUSD amount, USD is 1:1 after that fee, and ZAR cash-out uses the live (or fallback) USD/ZAR rate *without* the FX margin.

4.1 Worked example

Using seeded defaults and the configured fallback mid-market rate of 18.50 (the live API may differ; the algebra does not). For a R1,370.00 send, comparable to the RPW send size discussed by Rateweb (2026):

$$\begin{aligned} \texttt{transaction\_fee} &= 25.00 + 1370.00 \times 0.01 = 38.70 \\ \texttt{effective\_rate} &= 18.50 \times 1.02 = 18.87 \\ \texttt{net\_zar} &= 1370.00 - 38.70 = 1331.30 \\ \texttt{rlusd} &= 1331.30/18.87 = 70.551139 \end{aligned}$$

Estimated cash-out fee = $70.551139 \times 0.015 = 1.058267$ RLUSD; estimated recipient payout = 69.492872 RLUSD. The transaction fee (R38.70) is far below the R214–R217 Western Union / MoneyGram flat fees on that send size (Rateweb, 2026). That comparison is indicative only: our cash legs are simulated, we are not an agent network, and a live mid-market rate plus a 2% margin is not a licensed tariff.

A send whose fee consumes the principal is rejected. At the same defaults, R25.00 yields a R25.25 fee and a negative net, so the API returns 422.

Table 2 is the seeded `FeeConfig` an admin can edit at `PUT /admin/fee-config`. There is no fee-history table: the live row is what the next quote uses.

Table 2: Seeded fee parameters (fractions, not whole percents).

Field	Meaning	Default
`fixed_fee_zar`	Rand added to every send	25.00
`percentage_fee`	Share of send ZAR	0.0100
`fx_margin_percentage`	Added to mid-market rate	0.0200
`cash_out_fee_percentage`	Share of RLUSD at cash-out	0.0150

5 Exchange rates

The brief allows a public API, a mock service, or a configured table (University of Cape Town, 2026a). We use the first, with the third as fallback. `GET https://open.er-api.com/v6/latest/USD` supplies `rates.ZAR`. Successes are cached for 300 seconds (`EXCHANGE_RATE_CACHE_SECONDS`) so concurrent quotes do not stampede a free endpoint. Any failure (network, timeout, non-2xx, missing ZAR field) uses `USD_ZAR_RATE` from `.env` (default 18.50). There is no admin API for the fallback; changing it means editing the environment and restarting, and it only matters while the live API is down. Tests mock the live call except for dedicated exchange-rate tests.

6 Remittance limits

Tiers are derived from KYC, not stored on the user: approved → `verified`, otherwise `unverified`. Seeded caps match the brief's example: unverified R0 / R0 per day and month; verified R3,000 / R25,000. Admins may change caps via `PUT /admin/limit-tiers/{tier_key}`. Unverified is not

reachable through `POST /remittances` because that route requires approved KYC; the zero caps still exist so the example table is configurable.

Usage sums the sender's ZAR since the start of the UTC day or calendar month, with two exclusions that the original sketch did not have: `cancelled` never counts; `quoted` rows with `expires_at` already in the past never count. Unexpired quotes and every later live status—including `settlement_failed`—do count. That is deliberate: a failed on-chain payment still represents a confirmed cash-in that occupied limit capacity until an operator decides otherwise.

Table 3: Remittance statuses on one evolving row.

Status	**How we get there**	**Counts to limit?**
`quoted`	`POST /remittances`	Yes, until TTL
`cancelled`	sender cancel, still unexpired	No
`quoted` past TTL	clock; cash-in 409	No
`cash_in_pending`	sender cash-in	Yes
`cash_in_confirmed`	admin confirm	Yes
`settlement_queued`	enqueue after confirm	Yes
`settled`	worker Payment + credit	Yes
`settlement_failed`	worker error; no credit	Yes

7 Architecture

Figure 1 is the runtime. FastAPI serves JSON. SQLAlchemy talks to SQLite through Alembic revisions `0001_initial` and `0002_quote_session`. Redis is transport for settlement, not the source of truth. The settlement worker (the same function as `POST /admin/settlement/run` and `python -m scripts.run_settlement_worker`) signs Testnet Payments with `xrpl-py`. CORS is enabled for local UI origins (Vite 5173, React 3000, and the API host).

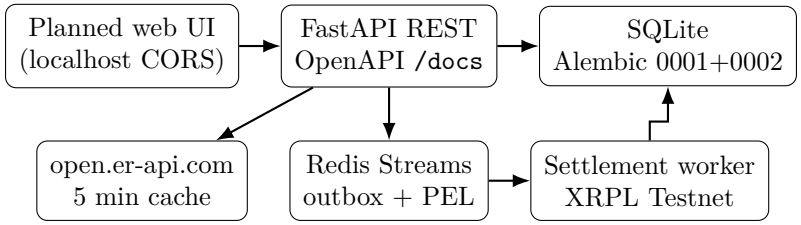

Figure 1: Main components. Redis carries settlement messages; durable status lives in SQLite.

Figure 2 is the money path the brief asks for.

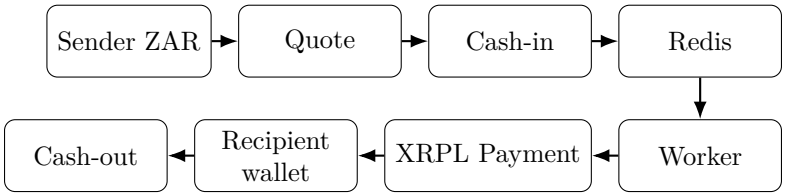

Figure 2: Settlement path: confirmed rand in, queued Payment, custodial UCTUSD, simulated fiat out.

Wallet design is a hybrid of the brief's two options (University of Cape Town, 2026a,b). One `PlatformWallet` holds scarce IOU liquidity. Each recipient still receives a real custodial Testnet

account, generated lazily on first settlement, so FR-style "individually attributable Payment + hash" is a genuine ledger transaction rather than a book entry that pretends to be one. Address and encrypted secret are committed *before* TrustSet. If an address exists but the trust line is not yet established, the worker reuses the stored secret; it does not mint a second wallet.

8 Cash-in, UCTUSD settlement, and cash-out

8.1 Cash-in

`POST /remittances/{id}/cash-in` is allowed only on an unexpired, non-cancelled `quoted` remittance. The sender chooses `agent_cash`, `bank_transfer`, or `card`. Status becomes `cash_in_pending`. Admin `confirm-cash-in` sets `cash_in_confirmed` and calls `enqueue_settlement`. No Payment is submitted at this step. The cash movement itself is simulated: there is no card acquirer, no agent till, and no bank confirmation webhook.

8.2 UCTUSD settlement

Enqueue inserts at most one `SettlementMessage` per remittance (unique constraint) and `XADD`s to stream `settlement_messages`. The Redis Stream ID is stored on `stream_entry_id`. If the database row exists but the stream ID is missing or the entry is gone, a later enqueue *republishes*—that covers "DB commit succeeded, Redis `XADD` failed". Redis is transport; the database row is source of truth.

The worker first reclaims its pending-entries list (`XREADGROUP` id `0`), then idle pending entries (`XAUTOCLAIM` on Redis 6.2+, else `XPENDING`+`XCLAIM`; local Memurai 5.0.14 has no `XAUTOCLAIM`), then new `>` entries. Entries are acknowledged only after the message is `completed` or `failed`. A crash mid-flight leaves the entry in the PEL for reclaim.

Before submitting a Payment, if `xrpl_settlement_tx_hash` is already set, the worker marks completed and **does not pay or credit again**. On the success path the Payment memo carries the remittance id; the hash, recipient credit, `settled`, and `completed` are one database commit. Admin retry accepts `failed`, `processing`, and `pending` (reset to pending, republish). `completed` returns 409.

8.3 Cash-out

The recipient must have approved KYC to cash out but may hold UCTUSD without it. Spendable `RecipientWallet.balance` is debited atomically at request time (`UPDATE ...WHERE balance >= :amt`). `GET /wallet/me` reports `balance_rlusd` and `spendable_balance` as that spendable figure (the slot is currently UCTUSD; official RLUSD later via `.env` if funded). `on_chain_balance` is spendable plus cash-out amounts in `requested` or `approved` only; it excludes completed cash-outs. Fiat payout is still simulated, but complete submits a Payment of the reserved UCTUSD from the recipient custodial account to the official UCTUSD issuer (burn/redeem). After a request, spendable drops while on-chain stays higher by the reserved amount until complete, when the on-chain figure drops too. A failed cash-out refunds spendable.

9 Database

Runtime schema is Alembic, not `create_all` alone. FastAPI lifespan runs `alembic upgrade head` then seeds default fees and limit tiers. Tests still use in-memory `create_all`. Main tables:

- `users` — name, unique email and mobile, bcrypt `password_hash`, role.
- `sessions` — `token_hash` (SHA-256 hex, unique), expiry, `revoked_at`. No plaintext token column.
- `kyc_applications` — one row per user; identification number encrypted.
- `beneficiaries` — payout profile; optional `linked_user_id`.
- `remittances` — full lifecycle on one row: amounts, fees, status (including `cancelled`), `tracking_ref`, `expires_at`, cash-in fields, XRPL hash.
- `settlement_messages` — one per remittance; status; `stream_entry_id`; attempts; failure reason.
- `recipient_wallets` — custodial address, encrypted secret, trust-line flag, spendable `balance`. No `on_chain_balance` column; that figure is computed on read.
- `platform_wallet` — singleton treasury address and encrypted secret.
- `cash_out_requests` — RLUSD amount, fiat, fee, payout, status, admin action stamps.
- `fee_config`, `limit_tiers` — singleton fees; unverified/verified caps.

Encrypted columns are stored as `VARCHAR(500)` (Fernet ciphertext). Existing SQLite files created only with `create_all` have no `alembic_version` table and will fail upgrade; demo databases should start empty and migrate.

A remittance points at one sender and one beneficiary. A settlement message points at one remittance. A recipient wallet is 1:1 with a user. Cash-out requests belong to the recipient user, not to a remittance row: the recipient may cash out aggregated RLUSD, not a single inbound Payment. Platform wallet is a singleton.

10 API overview

The API is JSON over HTTP with Bearer sessions. FastAPI emits Swagger UI at `http://127.0.0.1:8000/docs` and the OpenAPI schema at `/openapi.json`. Route prefixes: auth, users, kyc, beneficiaries, remittances, limits, wallet, cash-outs, and admin (fees, limits, settlement). CORS defaults allow local Vite (port 5173), local React (port 3000), and the API host on 8000, overridable with `CORS_ORIGINS` as a JSON list. Credentials are allowed.

Authz in short: customers act on their own resources; KYC-approved customers create quotes, beneficiaries, cash-in, and cash-outs; admins review KYC, confirm cash-in, run and retry settlement, action cash-outs, and edit fees and tiers. Tracking is visible to the sender, the linked recipient, or an admin (403 otherwise; 404 if the ref does not exist).

11 Security

Passwords are bcrypt via passlib and never stored in plaintext. Login returns a urlsafe bearer token once; the database stores only `SHA-256` of that token. Logout sets `revoked_at`. Opaque sessions were chosen over JWT because this is a single FastAPI process: logout must actually revoke access, which a signed JWT cannot do without a server-side denylist that looks like the sessions table anyway.

Two Fernet keys are required: `KYC_ENCRYPTION_KEY` for the identification number, `XRPL_KEY_ENCRYPTION_KEY` for platform and recipient XRPL secrets. A leak of one category does not decrypt the other. Keys live in environment variables, not in the database, not in API responses, not in application logs, and not in git (`.env` is local; `.env.example` has placeholders). XRPL secrets are decrypted only in the signing path. Bad ciphertext raises `RuntimeError` rather than returning empty values. Private keys are never printed by `setup_platform_wallet`. There is no custom application logger beyond uvicorn's access log (method/path/status, no bodies).

CORS is a demo convenience for a future SPA, not an authentication mechanism. Production would restrict origins, put keys in a secrets manager, rotate Fernet keys with a dual-decrypt window, and stop using SQLite as the vault for encrypted seeds.

12 Regulatory considerations

This section is an understanding of the problem, not a legal opinion (University of Cape Town, 2026a). A Testnet IOU does not make the operator a licensed money transmitter, and it does not take the corridor outside FICA or SARB rules.

KYC and AML. South African accountable institutions under FICA must identify customers and understand source of funds before providing money-remittance services. Our mock KYC collects the brief's fields, encrypts the ID number, and gates sending and cash-out on admin approval. That is a teaching model of a control, not a CDD programme: we do not screen against sanctions lists, we do not verify documents against Home Affairs, and we do not file suspicious-transaction reports.

Transaction monitoring (not fully automated). A real remittance business is expected to monitor velocity, structuring, mule patterns, and corridor risk. We enforce daily and monthly ZAR caps and keep a full remittance and settlement audit trail, including XRPL hashes. We do *not* ship an AML product: no rules engine, no alert queue, no SAR workflow. Operators would still need people and systems we have not built.

Custody of crypto assets. Recipient wallets are custodial: the platform generates Testnet keys, encrypts them, and signs on the customer's behalf. That is convenient and it is also a custody business. In production that would engage crypto-asset service-provider expectations (safekeeping, segregation, operational resilience) under the evolving South African CASP / FSCA framework. We store keys encrypted with a dedicated Fernet key; we do not offer self-custody withdrawal of the secret.

SARB and exchange control. Cross-border rand payments sit inside South African exchange-control and authorised-dealer arrangements. A stablecoin hop does not create a new legal corridor. Our cash-in is simulated, so we never actually take rand across the border. A live system would still need an authorised channel for the fiat legs and would still need to report what the SARB requires. RLUSD on Testnet is not a SARB-approved mechanism for discharging an exchange-control obligation.

Licensing. A commercial Western Union competitor in South Africa typically needs a FICA accountable-institution posture, Reserve Bank authorisation where the activity is a remittance or payment service, and—if offering crypto custody or exchange—the applicable FSCA crypto-asset authorisation. Agent cash-in/out may also engage payment-system and consumer-protection rules (clear fees, receipts, complaint handling). This coursework prototype is not licensed and must not be pointed at real customer funds or Mainnet credentials (University of Cape Town, 2026a).

Consumer protection and safeguarding follow from the same honesty: quoted fees should match charged fees (our quote locks the breakdown for 15 minutes); customer secrets should not leak (Section 11); and simulated balances must not be described as redeemable bank money. We do not hold client fiat in a trust account because we never collect real fiat. A production remittance UI would still need a pre-contract fee disclosure, a receipt with the tracking reference, and a complaints

path—none of which a Swagger page substitutes for. The planned screens in Section 15 are where those disclosures would live.

13 Assumptions and limitations

Assumptions we adopted on purpose: one customer identity for send and receive; admins only via `scripts/create_admin.py`; one KYC row with overwrite-on-resubmit until approved; beneficiary auto-match rather than invite-to-register; hybrid treasury plus per-recipient XRPL accounts; issuer address and currency code as `.env` so UCTUSD versus RLUSD is a config change (University of Cape Town, 2026b); Redis Streams as the queue; SQLite for development; quote TTL of 900 seconds.

Remaining limitations of *this* fork, not of the old sketch:

- SQLite's single-writer model is not a production remittance ledger.
- Cash-in and fiat cash-out are simulated; complete still burns reserved UCTUSD on-chain by paying the issuer.
- Automated tests mock XRPL (faucet, TrustSet, Payment). Redis is real (DB 15).
- Redis is required at runtime and in pytest.
- Login latency is dominated by bcrypt (see Section 14).
- No AML monitoring product, no sanctions screening, no production key-management service.

14 Performance summary

We cite Kerry-Lynn Whyte's Locust and settlement timings as a **baseline**, not as a re-run of this fork. The numbers were taken on 31 August and 1 September 2026 against a local single-process uvicorn and SQLite, and are recorded in `PERFORMANCE_TESTING.md`. They remain useful as an order-of-magnitude picture of the same FastAPI surface.

The HTTP mix (50 concurrent synthetic users, 60 s, 2,324 requests, zero failures) delivered about 39 req/s. Read endpoints sat in the high-single-digit milliseconds at the median; `POST /remittances` about 11 ms median. `POST /auth/login` was the outlier at about 400 ms median because bcrypt is deliberately expensive. Redis Stream enqueue measured about 517 messages/s for 200 publishes. Five real Testnet Payments averaged about 12.4 s each—ledger close time, not application CPU. We have not repeated Locust on the patched worker (PEL reclaim, outbox, hashed sessions). Relative bottlenecks—bcrypt on login, consensus on settlement—are unchanged in kind.

15 Planned web UI

The backend is API-complete; the brief still requires a web application (University of Cape Town, 2026a). This page is the screen map. The UI may be any SPA that speaks JSON (for example Vue on 5173 or React on 3000). CORS is already open for those origins. No framework is mandated here.

Table 4 is the screen map. Empty states matter: unverified senders see KYC pending, not a send form; expired quotes show "expired" not a cash-in button; a recipient without a Testnet account yet still sees a zero spendable wallet row.

Table 4: Planned screens mapped to existing endpoints (framework-agnostic).

Screen	What the user sees	Endpoints
Register / login	Name, email, mobile, password; logout	`POST /auth/register`, `/login`, `/logout`
Sender home	Profile, KYC badge, remaining daily/monthly ZAR	`GET /users/me`, `/kyc/me/status`, `/limits/me`
KYC form	Brief fields; resubmit if rejected	`POST /kyc`, `GET /kyc/me`
Beneficiaries	List and add; linked vs unlinked	`GET/POST /beneficiaries`
New quote	ZAR, beneficiary, locked fee/FX/RLUSD, 15 min timer, `MG` ref	`POST /remittances`
Quote actions	Cancel or choose cash-in method	`POST .../cancel`, `.../cash-in`
Track / history	Status, hash once settled	`GET /remittances/me`, `/remittances/track/{ref}`
Recipient wallet	Spendable vs on-chain, address, inbound list, cash-outs	`GET /wallet/me`
Cash-out	USD or ZAR amount; pending statuses	`POST /cash-outs`, `GET /cash-outs/me`
Admin KYC	Queue, approve, reject with reason	`GET /kyc`, `POST .../approve|reject`
Admin cash-in	Pending methods; confirm receipt	`GET /remittances`, `POST .../confirm-cash-in`
Admin settlement	Stream ids, retry stuck rows, run worker	`GET /admin/settlement`, `POST .../run`, `.../{id}/retry`
Admin cash-out	Approve, complete, or fail (refund)	`GET /cash-outs`, `POST .../approve|complete|fail`
Admin config	Fees and limit tiers	`GET/PUT /admin/fee-config`, `/admin/limit-tiers`

Admins are created with `scripts/create_admin.py`, never via public register. Demonstration until the UI exists: Swagger at `/docs` plus the settlement worker script. That is an API demo, not the marked web application.

References

FinMark Trust (2024). South africa to the rest of SADC remittances market assessment 2024. Technical report, FinMark Trust.

International Monetary Fund (2025). South africa: Technical assistance report—cross-border payments: South africa–zimbabwe corridor diagnostic. Technical Report 25/93, International Monetary Fund.

Rateweb (2026). Your transfer fee ignores where you're sending. that costs you. Online article analysing World Bank Remittance Prices Worldwide 2025 Q1 figures for South African outbound corridors. Secondary citation of World Bank RPW 2025 Q1. Reports Western Union and MoneyGram flat fees of about R214–R217 on a surveyed R1,370 send.

University of Cape Town (2026a). ECO5040W class project brief: XRPL-based FX remittance platform using RLUSD. Course brief, Financial Software Engineering.

University of Cape Town (2026b). ECO5040W project resources. Trust lines, IOUs, RLUSD vs fallback IOU, recommended stack.

World Bank (2025). Remittance prices worldwide. Technical report, World Bank. Quarterly survey of the cost of sending remittances. Primary source for corridor fee comparisons.